

```

"""
StegSim model.py - Core state objects
Agent state, system state, proposals, evaluation results.
Designed for compatibility with GCAT/BCAT formalism.
v1: pure mathematical agents, no LLM dependency.
"""

import hashlib
import json
import uuid
from dataclasses import dataclass, field, asdict
from typing import Optional

# — Hashing —————

def state_hash(obj: dict) -> str:
    canonical = json.dumps(obj, sort_keys=True, separators=(",", ":"))
    return "sha256:" + hashlib.sha256(canonical.encode()).hexdigest()

# — Agent state —————

@dataclass
class AgentState:
    """
    Lightweight agent state for v1 mathematical simulation.
    All values normalized to [0, 1].
    Compatible with GCAT (g,c,a,t) axis mapping.
    """
    agent_id: str
    trust: float # GCAT: t - trust metric
    authority: float # GCAT: c - control authority
    resources: float # GCAT: resource pressure
    alignment: float # GCAT: g - governance capacity proxy
    autonomy: float # GCAT: a - autonomous capability
    mutation_pressure: float # drift accumulator
    policy_snapshot_version: int # staleness tracker
    current_policy_version: int # global policy version at agent
    neighbors: list # agent_ids of graph neighbors

    def to_gcat(self) -> dict:
        """Map agent state to GCAT (g, c, a, t) vector."""
        return {

```

```

        "g": self.alignment,
        "c": self.authority,
        "a": self.autonomy,
        "t": self.trust,
    }

    def policy_lag(self) -> int:
        return max(0, self.current_policy_version - self.policy_snapshot_version)

    def policy_freshness(self, global_version: int) -> float:
        lag = max(0, global_version - self.policy_snapshot_version)
        return max(0.0, 1.0 - lag * 0.05)

    def to_dict(self) -> dict:
        return asdict(self)

    def hash(self) -> str:
        return state_hash(self.to_dict())

# — Environment state —————

@dataclass
class EnvironmentState:
    """
    Global environment state. Evolves each step via drift engine.
    """
    step: int
    resource_scarcity: float # 0=abundant, 1=critical
    trust_decay_rate: float # per-step trust loss
    authority_drift_rate: float # per-step authority staleness
    policy_version: int # current global policy version
    policy_lag: float # mean lag across agents
    mutation_pressure: float # global mutation pressure
    fragmentation: float # graph connectivity degradation
    communication_fidelity: float # 1=perfect, 0=none
    external_shock: float # episodic shock amplitude

    def to_dict(self) -> dict:
        return asdict(self)

    def hash(self) -> str:
        return state_hash(self.to_dict())

# — Viability margin —————

```

```

@dataclass
class ViabilityMargin:
    """
    Composite viability score  $V \in [0,1]$ .
     $V \geq 0.70$ : healthy
     $0.45 \leq V < 0.70$ : degraded but recoverable
     $0.25 \leq V < 0.45$ : warning zone
     $V < 0.25$ : inadmissible / fail-closed zone
    """
    trust_continuity: float
    authority_coherence: float
    resource_balance: float
    policy_freshness: float
    mutation_pressure: float
    fragmentation: float
    composite: float
    status: str # healthy|degraded|warning|inadmissible

    THRESHOLDS = {
        "healthy": 0.70,
        "degraded": 0.45,
        "warning": 0.25,
        "inadmissible": 0.0,
    }

    @classmethod
    def compute(cls, weights: dict, components: dict) -> "ViabilityMargin":
        w = weights
        c = components
        composite = (
            w["trust_continuity"] * c["trust_continuity"]
            + w["authority_coherence"] * c["authority_coherence"]
            + w["resource_balance"] * c["resource_balance"]
            + w["policy_freshness"] * c["policy_freshness"]
            - w["mutation_pressure"] * c["mutation_pressure"]
            - w["fragmentation"] * c["fragmentation"]
        )
        composite = max(0.0, min(1.0, composite))

        if composite >= 0.70:
            status = "healthy"
        elif composite >= 0.45:
            status = "degraded"
        elif composite >= 0.25:
            status = "warning"
        else:
            status = "inadmissible"

```

```

        return cls(
            trust_continuity=c["trust_continuity"],
            authority_coherence=c["authority_coherence"],
            resource_balance=c["resource_balance"],
            policy_freshness=c["policy_freshness"],
            mutation_pressure=c["mutation_pressure"],
            fragmentation=c["fragmentation"],
            composite=composite,
            status=status,
        )

    def to_dict(self) -> dict:
        return asdict(self)

# — System state —————

@dataclass
class SystemState:
    """
    Complete system state at one timestep.
    Agents + environment + viability.
    """
    step: int
    agents: dict # agent_id → AgentState
    environment: EnvironmentState
    viability: ViabilityMargin

    def agent_hash(self) -> str:
        summary = {aid: a.hash() for aid, a in self.agents.items()}
        return state_hash(summary)

    def full_hash(self) -> str:
        return state_hash({
            "step": self.step,
            "agent_hash": self.agent_hash(),
            "environment": self.environment.hash(),
            "viability": self.viability.composite,
        })

    def mean_trust(self) -> float:
        if not self.agents:
            return 0.0
        return sum(a.trust for a in self.agents.values()) / len(self.agents)

    def mean_authority(self) -> float:

```

```

        if not self.agents:
            return 0.0
        return sum(a.authority for a in self.agents.values()) / len(self.agents)

def mean_policy_freshness(self) -> float:
    if not self.agents:
        return 0.0
    gv = self.environment.policy_version
    return sum(a.policy_freshness(gv) for a in self.agents.values()) / len(se

# — Proposal —————

@dataclass
class Proposal:
    """
    A proposed transition from one agent at one step.
    Carries the action type and the agent's local state snapshot.
    """
    proposal_id: str
    step: int
    agent_id: str
    action_type: str # transfer_resources|update_policy|expand_author
    action_magnitude: float # normalized 0-1
    target_agent_id: Optional[str]
    local_state_snapshot: dict # agent's view of its own state at proposal time
    state_hash_before: str

    @classmethod
    def make(cls, step: int, agent: AgentState, action_type: str,
             magnitude: float, target: Optional[str], state_hash: str) -> "Propos
    return cls(
        proposal_id=f"proposal-{{step:08d}}-{{agent.agent_id}}",
        step=step,
        agent_id=agent.agent_id,
        action_type=action_type,
        action_magnitude=magnitude,
        target_agent_id=target,
        local_state_snapshot=agent.to_dict(),
        state_hash_before=state_hash,
    )

    def to_dict(self) -> dict:
        return asdict(self)

# — Evaluation result —————

```

```

@dataclass
class EvaluationResult:
    """
    Result of governance evaluation for one proposal.
    Decision vocabulary: ALLOW | DENY | FLAG | FAIL_CLOSED
    """
    proposal_id: str
    step: int
    agent_id: str
    decision: str
    reason: str

    # Gate results (all must pass for ALLOW)
    local_policy_valid: bool
    authority_valid_current_state: bool
    viability_preserved: bool
    legitimacy_capacity_ok: bool

    # Quantitative signals
    viability_before: float
    viability_after_projected: float
    viability_delta: float
    authority_staleness: float # policy_lag / max_acceptable_lag
    legitimacy_capacity: float # Lambda(x)
    autonomy_projected: float # a after transition
    local_global_disagree: bool # local passed, global failed

    # Hashes
    state_hash_before: str
    state_hash_after_projected: str

    def to_dict(self) -> dict:
        return asdict(self)

# — Transition classes (v3 periodic table seed) —————

TRANSITION_CLASSES = {
    "T-101": {"name": "informational_mutation", "consequence": "low", " "
    "T-201": {"name": "resource_transfer", "consequence": "medium", " "
    "T-221": {"name": "resource_redistribution", "consequence": "medium", " "
    "T-301": {"name": "trust_update", "consequence": "medium", " "
    "T-311": {"name": "trust_fragmentation", "consequence": "high", " "
    "T-401": {"name": "authority_assertion", "consequence": "high", " "
    "T-402": {"name": "authority_escalation", "consequence": "critical", " "
    "T-501": {"name": "policy_update", "consequence": "medium", " "

```

```
"T-511": {"name": "policy_lag_accumulation", "consequence": "high", "
"T-550": {"name": "irreversible_collapse", "consequence": "terminal", "
"T-601": {"name": "recovery_action", "consequence": "positive", "
"T-000": {"name": "idle", "consequence": "none", "
}
```

```
ACTION_TO_TRANSITION_CLASS = {
    "idle": "T-000",
    "transfer_resources": "T-201",
    "update_policy": "T-501",
    "expand_authority": "T-401",
    "assert_trust": "T-301",
    "recover": "T-601",
}
```